

Technical Assessment: Puzzle Solver

Goal

Write a client that solves as many puzzles as possible in 30 seconds.

Description

Each puzzle is a sequence of words split into pieces. Your client collects the pieces from the server, puts them back in order and submits the solution. Your score is the number of puzzles you solve correctly within the 30-second window.

Server Setup (via Docker)

Run the puzzle server locally with Docker:

1. Run it:

```
docker run -p 8080:8080 ifajardov/puzzle
```

The server listens on <http://localhost:8080>.

API description

- Starts a new puzzle.

Python

```
### GET /start
```

```
{ "session_id": "b34c5dde4cf6aa92d2e960682e8c73ac" }
```

session_id: identifies the puzzle in all later calls.

- Get puzzle piece

Python

```
### GET /get/{n}?session_id={session_id}
```

```
{ "id": 28, "word": "jacket" }
```

Where:

n: any non-negative integer (0, 1, 2, ...). The same n always returns the same piece within a session.

- Submit a solution.

Python

```
### POST /submit
```

```
{ "session_id": "b34c5dde4cf6aa92d2e960682e8c73ac", "words":  
["garden", "violet", "jacket"] }
```

```
### Response
```

```
{ "ok": true }
```

Where:

ok: true: the puzzle is solved and its session is closed.

ok: false: the solution is wrong. The session stays open, so you can keep fetching pieces and submit again.

Submission Instructions

1. Push your solution to a **GitHub repository**
2. Include a **README.md** with:
 - How to run your solution
 - Your strategy for speed and correctness
 - What were the challenges to solve the puzzle
3. **Send the GitHub link by email** to the person who provided you this challenge